

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3069 – Computación Paralela

Sección 10

Ing. Kimberly Barrera



Proyecto 1

Angie Nadissa Vela López, 23764

Javier Alexander Linares Chang, 231135

Roberto Emiliano Camposeco Torres, 23968

GUATEMALA, 09 de septiembre de 2026

Índice

Índice.....	2
Introducción	4
Antecedentes.....	4
Programación en memoria compartida y OpenMP.....	4
El método de diseño PCAM.....	4
SDL2 como librería gráfica	4
Speedup, eficiencia y la Ley de Amdahl	5
Cuerpo del informe	5
1. Descripción general del proyecto	5
2. Metodología, PCAM aplicado al problema	6
Partición	6
Comunicación	6
Aglomeración.....	6
Mapeo.....	6
3. Arquitectura del software	6
4. Implementación	7
4.1 Movimiento y corrección del rebote en bordes.....	7
4.2 Detección y resolución de choques	7
4.3 Versión secuencial (v1).....	7
4.4 Versiones paralelas con OpenMP (v2, v3, v3-naive, v4)	7
4.5 Programación defensiva.....	8
5. Resultados y análisis de desempeño	8
5.1 Escalabilidad, speedup y eficiencia según N.....	9
5.2 Efecto de la política de reparto (schedule).....	9
5.3 Costo de sincronizar de más, v3 frente a v3-naive	9
5.4 Verificación de corrección, checksum entre versiones.....	10
5.5 El render sigue siendo secuencial, Ley de Amdahl	10
Conclusiones	10
Recomendaciones	11
Referencias bibliográficas.....	11
Apéndice, Material suplementario	12
Documentos del repositorio	12
Comandos usados para generar las mediciones de este informe.....	12
Repositorio de código	12
Anexo 1, Diagrama de flujo del programa	12

Anexo 2, Catálogo de funciones.....	15
neon_lines.h y neon_lines.cpp, lógica pura del screensaver	15
config.h y config.cpp, parámetros y validación	16
paralelo.cpp, versiones con OpenMP	16
render.h y render.cpp, dibujo y HUD	16
bench_io.h, bench_io.cpp y main(), utilidades compartidas y punto de entrada	17
Anexo 3, Bitácora de pruebas.....	17
Verificación de corrección, checksum entre versiones (N=2000)	18
Verificación del rebote en esquinas.....	18
N máximo que sostiene 60 FPS o más (modo ventana, con render real)	18
Capturas de las mediciones.....	18

Introducción

Este informe cuenta como disenamos, implementamos y medimos el rendimiento de Neon Lines, un screensaver con estetica Tron que hicimos en C++ con SDL2 y paralelizamos con OpenMP para el Proyecto 1 de Computacion Paralela y Distribuida (CC3069, UVG). N lineas de color neon se mueven por la ventana, rebotan en los bordes con fisica de reflexion real y, cuando dos chocan entre si, intercambian su velocidad (choque elastico) y su color, como los light-cycles de la pelicula.

Mas alla de que el programa funcione, quisimos mostrar el proceso completo que pide el curso: partimos de una version secuencial medible, aplicamos el metodo PCAM, armamos varias versiones con OpenMP probando distintas formas de sincronizar, y medimos la mejora real con speedup y eficiencia. Tambien veniamos de dos prototipos con problemas distintos, que este informe explica como corregimos (seccion 3).

Antecedentes

Programación en memoria compartida y OpenMP

En memoria compartida, varios hilos usan el mismo espacio de memoria y se reparten el trabajo sin mandarse mensajes explicitos (a diferencia de la memoria distribuida). OpenMP es el estandar mas usado para esto en C, C++ y Fortran: en vez de manejar hilos a mano como con pthreads, se agregan directivas del compilador (`#pragma omp`). La especificacion oficial lo define como “a base program that is annotated with OpenMP directives or that calls OpenMP API runtime library routines” (OpenMP Architecture Review Board, 2021); un programa secuencial normal al que se le agregan anotaciones que el compilador puede usar para generar codigo multihilo, o simplemente ignorar. Eso es justo lo que permite ir pasando de secuencial a paralelo poco a poco.

OpenMP nos resolvio dos cosas: repartir un bucle entre hilos (`parallel for`, `schedule`) y coordinar el acceso a datos compartidos (`critical`, `atomic`, `barrier`, `single`, `reduction`). Estos mecanismos se explican en la seccion de Implementacion.

El método de diseño PCAM

Para decidir que paralelizar y como, seguimos el metodo PCAM de Ian Foster (Foster, 1995): Particion (partir el computo en tareas chiquitas), Comunicacion (ver que datos hay que compartir entre tareas y cuando), Aglomeracion (agrupar tareas chiquitas cuando coordinarlas por separado sale mas caro que el beneficio) y Mapeo (repartir esas tareas entre los hilos, buscando parejo el trabajo). La seccion 2 del cuerpo aplica estas cuatro etapas al problema del screensaver.

SDL2 como librería gráfica

Usamos SDL2 (Simple DirectMedia Layer 2), una libreria en C que abstrae la creacion de ventanas, el dibujo de primitivas 2D con aceleracion por hardware y la medicion de tiempo de alta resolucion (Lantinga y contribuidores de SDL, 2024). La elegimos sobre OpenGL porque el problema no necesita un pipeline 3D, y porque SDL no es thread-safe, algo que en realidad nos ayudo: obliga a

separar la parte que se puede paralelizar (la física) de la que tiene que quedarse secuencial (el dibujo), justo el patron que pide la Fase 3 del proyecto.

Speedup, eficiencia y la Ley de Amdahl

El speedup de una version con p hilos es $S(p) = T1 / T_p$, y la eficiencia es $E(p) = S(p) / p$. Amdahl (1967) demostro que el speedup maximo esta limitado por la fraccion del programa que no se puede paralelizar: si una fraccion f del tiempo es secuencial si o si, el speedup nunca pasa de $1/f$. Aqui la Fase 3 (el render) siempre es secuencial, y por eso le pone un techo al speedup del frame completo, como se ve en Resultados.

Cuerpo del informe

1. Descripción general del proyecto

Neon Lines es un screensaver de haces de luz neon: N líneas rectas de largo fijo se mueven por una ventana oscura de al menos 640×480 pixeles, rebotan de forma especular en los bordes y, cuando dos se cruzan, intercambian su velocidad (choque elastico real, con cambio de direccion) y su color. La paleta sigue la estetica Tron: cian (~55%), naranja (~25%), azul electrico (~13%) y ambar (~7%). La tabla 1 resume como cumplimos cada requisito del enunciado.

Requisito del enunciado	Cómo se cumple
Parámetro N	Argumento posicional N , validado en $[1, 100000]$; se solicita por consola si se omite.
Varios colores pseudoaleatorios	4 familias de tono elegidas con <code>std::mt19937</code> sembrado por <code>-seed</code> .
$\text{Canvas} \geq 640 \times 480$	800×600 por defecto; <code>--width/--height</code> configurables, con mínimo forzado.
Movimiento y estética interesante	Estela neón, rejilla de fondo, glow aditivo (SDL <code>BLENDMODE_ADD</code>).
Física / trigonometría	Rebote especular, choque elástico, intersección de segmentos por producto cruz.
Interacciones entre los N elementos	Detección de colisión entre cada par de líneas (grilla espacial en <code>v4</code>), con intercambio de color y velocidad.
Despliegue de FPS	En el título de la ventana, en consola y en un HUD con <code>SDL2_ttf</code> .
Versión secuencial y paralela(s)	<code>v1</code> (secuencial) y cuatro variantes paralelas con OpenMP (sección 4).
Cálculo de speedup	Modo benchmark con <code>--csv</code> ; <code>scripts/analizar.py</code> calcula speedup y eficiencia (sección 5, Anexo 3).

Tabla 1. Cumplimiento de los requisitos del enunciado del Proyecto 1.

2. Metodología, PCAM aplicado al problema

Aplicamos las cuatro etapas de PCAM al ciclo de un frame.

Partición

Partimos el trabajo bien fino, en dos niveles: una tarea por línea para el movimiento (N tareas independientes) y una tarea por cada par (i, j) para la detección de colisiones ($N*(N-1)/2$, o solo pares vecinos en $v4$). Partir fino nos dejó la decisión de cuanto agrupar para la etapa de Aglomeración, en vez de decidirlo desde el principio.

Comunicación

El movimiento no necesita comunicación entre tareas. La detección si necesita leer posiciones ya actualizadas, por eso hay una barrera entre la Fase 1 y la Fase 2. Resolver un choque escribe en dos líneas a la vez, así que centralizamos esa escritura en un solo hilo por frame en vez de usar candados por línea, porque los choques por frame son pocos comparados con la cantidad de líneas.

Aglomeración

El reparto de las N tareas de movimiento lo hace `#pragma omp for` según el schedule elegido. Para colisiones, cada hilo guarda lo que encuentra en un buffer local y lo fusiona una sola vez ($v3$), en vez de sincronizar cada hallazgo por separado ($v3$ -naive): es aglomerar sincronización, no solo cómputo.

Mapeo

Dejamos que OpenMP reparta con `schedule(runtime)`, configurable con `--schedule` entre `static`, `dynamic` y `guided`. Esto importa porque la Fase 2 recorre un bucle triangular (la fila i tiene $N-i-1$ pares), así que un reparto estático deja el trabajo desbalanceado entre hilos; la sección 5.2 mide ese efecto.

3. Arquitectura del software

El código está separado en módulos con responsabilidades bien definidas; la versión secuencial y las paralelas comparten exactamente la misma lógica y solo cambia como recorren el arreglo de líneas, lo que nos permitió comparar un checksum final entre versiones para verificar que todas dan el mismo resultado (sección 5.4).

Módulo	Responsabilidad
config.h / config.cpp	Parámetros de línea de comandos y validación defensiva; solicitud interactiva de N .
neon_lines.h / neon_lines.cpp	Lógica pura, sin SDL; struct LightBeam, paleta, movimiento, rebote, choques, checksum.
render.h / render.cpp	Dibujo con SDL2 en lote, rejilla, chispas y HUD con SDL2_ttf.
bench_io.h / bench_io.cpp	Creación de ventana/rendering y escritura del CSV, compartidas por ambos ejecutables.
secuencial.cpp	main de la versión v1 (un solo hilo).
paralelo.cpp	main de v2 / v3 / v3-naive / v4, seleccionables con <code>--mode</code> .

Tabla 2. Módulos del código fuente y su responsabilidad.

Esta arquitectura junta lo mejor de dos prototipos que ya teníamos (diagnostico completo en PLAN_DE_CORRECCION.md): uno tenía la física de colisiones bien pero un bug de rebote en las esquinas, y el otro tenía mejor arquitectura pero sin colisiones ni paralelización real. La versión final se quedó con la arquitectura del segundo y le puso, ya corregida, la física del primero.

4. Implementación

4.1 Movimiento y corrección del rebote en bordes

Cada línea se representa con centro, ángulo, largo y velocidad, y su posición se integra con $new_x = x + vx \cdot dt$. El bug del prototipo original invertía la velocidad en cada frame en que un extremo seguía fuera de la ventana, sin fijarse si la línea ya iba de regreso, y eso la hacía oscilar sin parar; en las esquinas se quedaba atrapada. La corrección invierte la velocidad de un eje solo si la línea todavía va hacia esa pared.

- Si $(cx - hx < 0)$ y $vx < 0$, invertimos vx ; si vx ya es positivo, no lo tocamos. Misma regla, aparte, en el eje Y .

Lo verificamos con un programa de prueba que registra la trayectoria frame a frame (Anexo 3): un rebote de esquina cambia los dos ejes en un solo frame y no se repite después.

4.2 Detección y resolución de choques

La detección primero filtra por cajas delimitadoras (AABB) y, si se solapan, corre un test de intersección de segmentos por orientación; es de solo lectura, por eso es segura para paralelizar. La resolución si escribe en ambas líneas: si todavía se están acercando, aplica un choque elástico entre masas iguales e intercambia el color de las dos, para no resolver el mismo cruce varias veces seguidas.

4.3 Versión secuencial (v1)

Recorre las N líneas con un for normal para el movimiento y un doble for ($i < j$) para las $N \cdot (N-1)/2$ parejas en la detección; los choques se resuelven en el mismo orden en que se encontraron. Es el punto de comparación (T_1) para todos los cálculos de speedup.

4.4 Versiones paralelas con OpenMP (v2, v3, v3-naive, v4)

Las cuatro comparten el mismo código de movimiento y resolución de v_1 , solo cambia como se reparte el trabajo entre hilos (tabla 3).

Versión	Fase 1 (movimiento)	Fase 2 (detección)	Resolución	Vértices
v1	for secuencial	doble for secuencial	secuencial	secuencial
v2	omp parallel for	secuencial (igual a v1)	secuencial	secuencial
v3	omp parallel for	omp for + buffer local + critical de fusión	omp single, orden (i,j)	omp parallel for
v3-naive	omp parallel for	omp for + critical por cada par	omp single, orden (i,j)	omp parallel for
v4	omp parallel for	grilla espacial, omp for por celda + 8 vecinas	omp single, orden (i,j)	omp parallel for

Tabla 3. Estrategia de paralelización de cada versión.

Mecanismos de sincronizacion que usamos, y por que:

- Barrera implicita al final de omp for: garantiza que las posiciones ya esten actualizadas antes de leerlas en la Fase 2.
- Buffer local por hilo mas critical de fusion (v3, v4): un candado, una sola vez por hilo, en vez de uno por cada hallazgo.
- critical por cada par (v3-naive): variante a proposito menos eficiente, la usamos de comparacion.
- barrier explicita: asegura que todas las fusiones ya terminaron antes de resolver los choques.
- single: resuelve los choques en un solo hilo, porque cada choque escribe en dos lineas y poner candados por linea haria perder el orden deterministico sin ganancia real.
- reduction(+:pairsTested): suma un contador por hilo sin necesitar candado.
- schedule(runtime), configurable con --schedule: importa por la forma triangular de la Fase 2 (seccion 5.2).

v4 le agrega particion espacial: una grilla uniforme donde cada celda mide al menos el rectangulo mas grande que puede ocupar una linea, asi que dos lineas que puedan tocarse siempre caen en celdas vecinas. Cada hilo solo prueba su celda y las 8 de al lado, asi que el trabajo baja de $O(N^2)$ a algo mucho mas cercano a $O(N)$.

4.5 Programación defensiva

Validamos todos los argumentos: N entre 1 y 100000 (con strtol, no atoi), minimos de ventana forzados, banderas exclusivas de la version paralela rechazadas en el binario secuencial, y cualquier valor mal formado termina con un mensaje especifico y codigo de error. Si falta N y no es benchmark, lo pedimos por consola y reintentamos ante entradas invalidas. La inicializacion grafica tambien es defensiva: si falla el renderer acelerado, reintentamos por software, y si la fuente TTF no abre, sigue sin HUD en vez de terminar.

5. Resultados y análisis de desempeño

Todas las mediciones se hicieron en modo --headless (computo puro, Fases 1+2, la parte que se paraleliza), con la misma semilla (--seed 42) y al menos 10 repeticiones por configuracion (varias combinaciones para N=2000 tienen entre 11 y 32, por corridas adicionales de schedule y checksum). El detalle completo esta en el Anexo 3.

Parámetro	Valor
CPU	AMD Ryzen 5 3550H with Radeon Vega Mobile Gfx, 4 nucleos fisicos / 8 hilos logicos, 2.1 GHz base
RAM / SO	12 GB / Windows 11 Home, 64 bits (build 10.0.22H2) (build 10.0.22H2)
Compilador	g++ (GCC) 11.2.0, MinGW-w64, hilos POSIX-SEH, -O2 -fopenmp -Wall -Wextra (sin advertencias)
Semilla / frames	--seed 42 / --frames 600 (matriz principal), --frames 180 (ventana real)

Tabla 4. Configuración de la máquina de pruebas.

5.1 Escalabilidad, speedup y eficiencia según N

La tabla 5 muestra una seleccion representativa de speedup ($T1/Tp$) y eficiencia (speedup/hilos); el conjunto completo esta en el Anexo 3, Captura A3.1.

N	Versión	Hilos	Speedup	Eficiencia
100	v3	8	0.11	0.014
100	v4	8	0.12	0.015
500	v4	8	1.26	0.158
1000	v4	8	4.03	0.504
2000	v3	8	3.40	0.425
2000	v4	4	12.73	3.183
2000	v4	8	7.35	0.918
4000	v3	8	3.21	0.401
4000	v4	8	13.15	1.643

Tabla 5. Speedup y eficiencia, muestra representativa.

Con N chico (100), el costo fijo de sincronizacion supera el trabajo disponible y las dos versiones paralelas resultan mas lentas que v1 (speedup 0.11 y 0.12). A partir de N=500 el speedup ya es positivo y crece con N, llegando a 13.15x (v4, N=4000, 8 hilos). La eficiencia baja segun sube N en la mayoria de los casos, el resultado esperado de repartir un costo fijo de sincronizacion entre mas hilos.

Un dato curioso: en v4 con N=2000, usar 4 hilos (12.73x) resulto mejor que usar 8 (7.35x), porque la grilla espacial deja tan poco trabajo por frame que mas hilos solo agregan overhead. Aun asi, v4 le gana por mucho a v3 en N grande (13.15x frente a 3.21x en N=4000): la mejora algoritmica de evitar las N^2 parejas pesa mas que la paralelizacion sola.

5.2 Efecto de la política de reparto (schedule)

Schedule	Tiempo total (ms)	FPS	Speedup vs. v1
dynamic	2.271	447.9	3.40
guided	2.709	370.1	2.85
static	2.863	351.1	2.70

Tabla 6. Schedule, v3, N=2000, 8 hilos.

dynamic fue ~19% mas rapido que guided y ~26% mas rapido que static, confirmando la hipotesis: la Fase 2 es un bucle triangular (la fila i tiene N-i-1 pares), asi que un reparto estatico deja a unos hilos con mucho mas trabajo que a otros sin reequilibrio, mientras dynamic redistribuye bloques chicos segun cada hilo va terminando.

5.3 Costo de sincronizar de más, v3 frente a v3-naive

Modo	N	Hilos	Tiempo (ms)	Speedup
v3 (buffer local por hilo)	2000	8	2.271	3.40
v3-naive (critical por par)	2000	8	2.411	3.20
v3 (buffer local por hilo)	4000	8	6.394	3.21
v3-naive (critical por par)	4000	8	7.623	2.69

Tabla 7. v3 frente a v3-naive, 8 hilos.

v3-naive resulto entre 6% y 19% mas lento que v3, y la brecha crece con N porque crecen los choques por frame: cada hallazgo individual en v3-naive dispara una seccion critica, mientras que v3 solo paga ese costo una vez por hilo al fusionar su buffer local. Ilustra el principio de Aglomeracion aplicado a sincronizacion, no solo a computo.

5.4 Verificación de corrección, checksum entre versiones

Cada corrida de benchmark calcula un hash FNV-1a de posiciones y colores finales. Con la misma semilla y frames, probamos 25 combinaciones de version, hilos y schedule para N=2000 y todas dieron exactamente el mismo checksum, 5186230241032989736; lo mismo pasa para N=100, 500, 1000 y 4000 (detalle en Anexo 3). Ninguna estrategia de paralelizacion introduce una condicion de carrera: el estado final es identico bit a bit, sin importar hilos ni reparto.

5.5 El render sigue siendo secuencial, Ley de Amdahl

La Fase 3 (render SDL) siempre es secuencial. Medimos el frame completo con ventana real en esta maquina y encontramos que v1, v3 y v4 sostienen 60 FPS o mas hasta aproximadamente N=1200-1300 lineas, practicamente lo mismo entre las tres versiones (detalle en el Anexo 3). Esto es la Ley de Amdahl en la practica: aunque v4 hace la deteccion de colisiones muchisimo mas barata que v1 (en headless, a N=2000, v4 tarda una fraccion de lo que tarda v1 en la Fase 2), el render (Fase 3) cuesta entre 14 y 19 ms por frame en este rango de N, mucho mas que cualquier fase paralelizable, asi que el techo de FPS con ventana termina siendo casi el mismo para las tres.

Conclusiones

- Diseñamos e implementamos un screensaver funcional (Neon Lines) que cumple todos los requisitos del enunciado: parametro N, colores pseudoaleatorios, canvas de al menos 640x480, movimiento con fisica real (rebote especular y choque elastico entre lineas), interaccion entre los N elementos, y FPS en pantalla, titulo y consola.
- Implementamos una version secuencial (v1) y cuatro paralelas con OpenMP (v2, v3, v3-naive, v4), todas sobre la misma logica, y verificamos que ninguna introduce condiciones de carrera: las cinco dan el mismo checksum final en las 25 combinaciones de hilos y schedule que probamos para N=2000 (y en N=100, 500, 1000 y 4000 tambien).
- Medimos speedup y eficiencia con al menos 10 repeticiones por configuracion, para N entre 100 y 4000 lineas. El comportamiento es el esperado: con N chico el costo de sincronizacion supera el trabajo disponible y las versiones paralelas salen mas lentas que v1, mientras que desde N=500 el speedup crece de forma sostenida, llegando hasta 13.15x con la particion espacial (v4) en N=4000.
- La particion espacial (v4) fue la mejora mas grande de todas las que probamos, mas que cuadruplicando el speedup de v3 en N=4000 (13.15x frente a 3.21x), lo que confirma que una mejora algoritmica (reducir cuantos pares se evaluan) puede superar ampliamente a una mejora puramente de paralelizacion sobre el mismo algoritmo $O(N^2)$.
- La politica de reparto importa: dynamic le gano a static y guided en la deteccion de colisiones (hasta 26% mas rapido), consistente con la forma triangular y desbalanceada de ese bucle. De la misma forma, agrupar los hallazgos en un buffer local antes de sincronizar (v3) fue consistentemente mas rapido que sincronizar en cada hallazgo (v3-naive).

- El render (Fase 3) es secuencial en las cinco versiones, una restriccion real de SDL2 y no algo evitable, y por la Ley de Amdahl le pone un limite al speedup del frame completo que no se supera agregando mas hilos al computo: lo confirmamos midiendo con ventana real, donde las tres versiones sostienen 60 FPS hasta un N muy similar (~1200-1300).

Recomendaciones

1. Medir tambien el frame completo (Fases 1+2+3) en modo ventana para v2, no solo para v1, v3 y v4, para completar el panorama de la Ley de Amdahl con la version paralela mas simple del proyecto.
2. Ampliar la matriz de mediciones a valores de N mas grandes (8000, 16000) para confirmar si la ventaja de v4 sobre v3 sigue creciendo o si el costo fijo de mantener la grilla espacial eventualmente la alcanza.
3. Explorar una variante de v4 donde el conteo por celda de la grilla tambien se paralelice (hoy es secuencial porque sale barato frente al filtrado que reemplaza), para ver si aporta algo medible con N muy grande.
4. Grabar un GIF corto para la presentacion oral, mostrando en camara lenta el intercambio de color cuando dos lineas chocan; ya tenemos la captura estatica en este informe (Anexo 3), pero el movimiento se aprecia mejor en video.
5. Correr la matriz completa de benchmark.ps1 sin acotar por parametros al menos una vez mas antes de cualquier entrega futura, para tener cobertura total de N por hilos por version por schedule (en este informe usamos un subconjunto representativo de 276 corridas reales, no la matriz completa).

Referencias bibliográficas

OpenMP Architecture Review Board. (2021). OpenMP Application Programming Interface, Version 5.2. <https://www.openmp.org/spec-html/5.2/openmp.html>

Foster, I. (1995). Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering. Addison-Wesley. Disponible en <https://www.mcs.anl.gov/~itf/dbpp/>

Lantinga, S., y contribuidores de SDL. (2024). Simple DirectMedia Layer (SDL2), documentación oficial. <https://wiki.libsdl.org/SDL2/FrontPage>

Pacheco, P. S., y Malensek, M. (2022). An Introduction to Parallel Programming (2.^a ed.). Morgan Kaufmann.

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Conference Proceedings, 30, 483-485.

Apéndice, Material suplementario

Esta sección reúne referencias a material adicional que respalda este informe pero que, por su extensión, no se incluyó completo en el cuerpo del documento.

Documentos del repositorio

- PLAN_DE_CORRECCION.md, diagnóstico detallado de los dos prototipos previos del equipo y el plan de trabajo que guió la versión final del código.
- README.md, instrucciones de compilación, ejecución, parámetros de línea de comandos y estructura del proyecto.
- docs/01_Seleccion_Herramienta_Grafica.md, justificación de la elección de SDL2 sobre OpenGL para este proyecto.
- resultados/bitacora.csv y resultados/resumen.csv, datos crudos y resumen de todas las mediciones de desempeño.
- scripts/benchmark.ps1 y scripts/analizar.py, automatización de la matriz de mediciones y del cálculo de speedup y eficiencia.

Comandos usados para generar las mediciones de este informe

Ejecutados desde la raíz del repositorio, con el ejecutable ya compilado con make all.

```
& "scripts/benchmark.ps1" -Ns 100,500,1000,2000,4000 -Threads 8 -Modes v3,v4 -Schedules dynamic -Repeats 10
```

```
& "scripts/benchmark.ps1" -Ns 2000 -Threads 8 -Modes v3 -Schedules static,dynamic,guided -Repeats 10 ; -Ns 2000,4000  
-Modes v3,v3-naive -Repeats 10 ; -Threads 4,8 -Modes v2,v3,v3-naive,v4 -Schedules static,dynamic,guided -Repeats 1  
(para la tabla de checksums)
```

```
python scripts/analizar.py resultados/bitacora.csv --out resultados/resumen.csv
```

Repositorio de código

El código fuente completo, con el historial de control de versiones del equipo, está disponible en el repositorio de GitHub del proyecto, privado hasta el momento de la evaluación, según pide el enunciado.

Anexo 1, Diagrama de flujo del programa

El diagrama se presenta en dos partes, el flujo general del programa (inicio, inicialización, loop principal y cierre) y el detalle de lo que ocurre dentro de un único frame del loop principal (las tres fases del algoritmo). Ambos diagramas señalan explícitamente los seis elementos que pide la rúbrica, captura de argumentos, solicitud de ingreso de datos, programación defensiva, secciones paralelas, mecanismos de sincronía y despliegue de resultados.

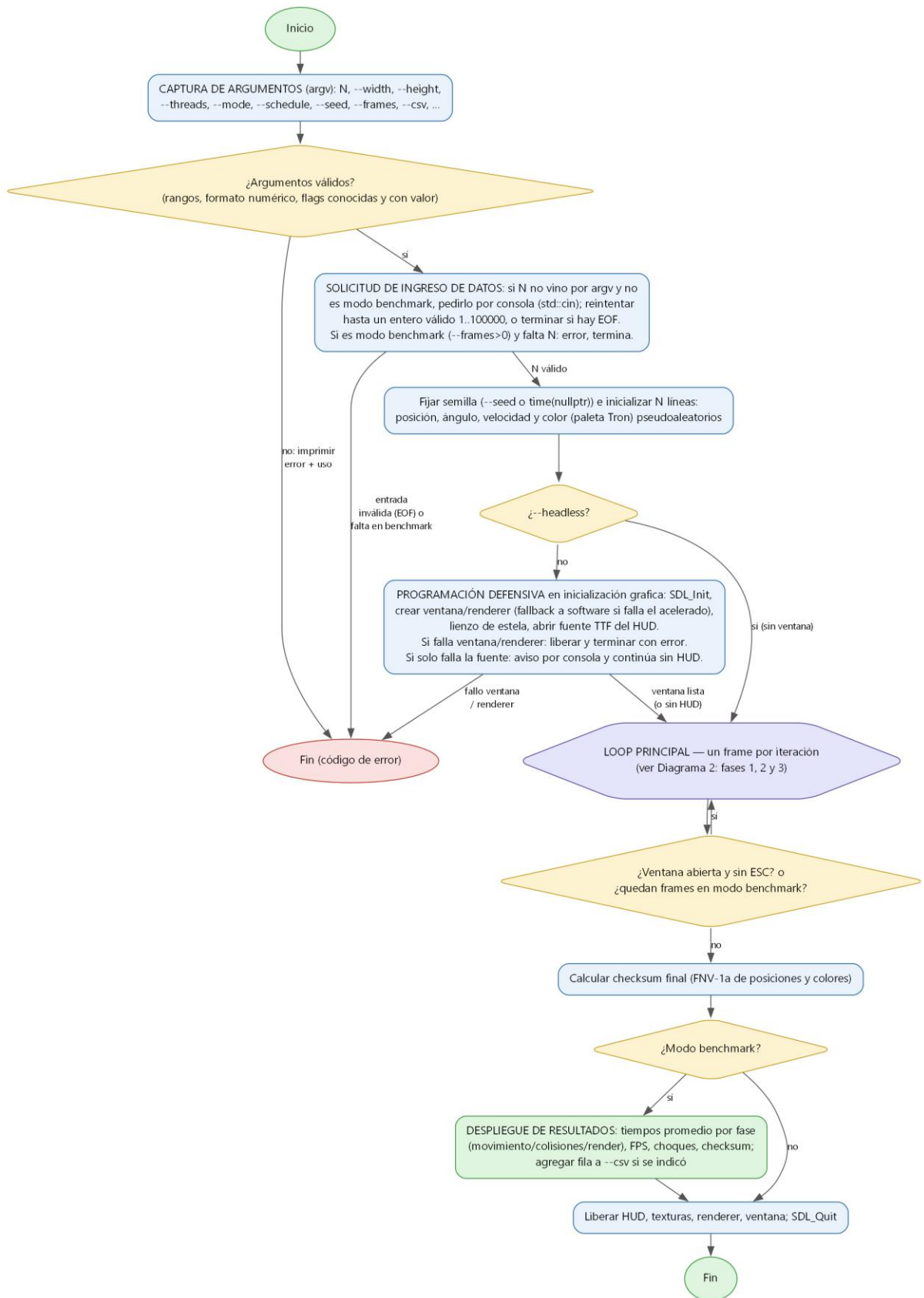


Diagrama 1. Flujo general, inicio, validación de argumentos, inicialización y cierre.

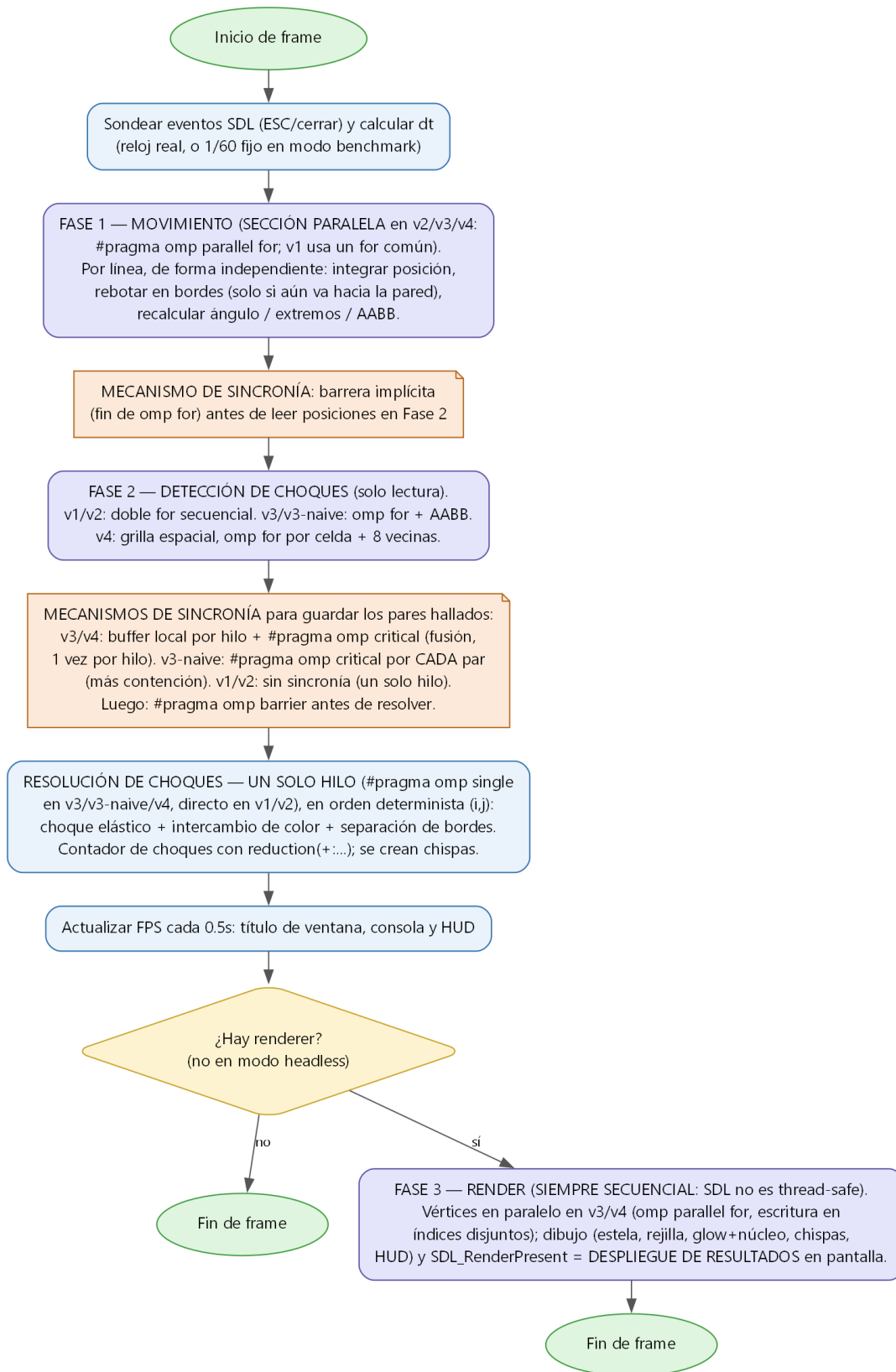


Diagrama 2. Detalle de un frame del loop principal, fases 1 (movimiento), 2 (colisiones) y 3 (render), con sus mecanismos de sincronización.

Anexo 2, Catálogo de funciones

Se documentan las funciones más relevantes de cada módulo, con sus entradas (nombre, tipo, uso), salidas (nombre, tipo, uso) y el propósito de cada una. El detalle completo de las funciones auxiliares internas está comentado directamente en el código fuente, en docs/03_Catalogo_de_funciones.md dentro del repositorio.

neon_lines.h y neon_lines.cpp, lógica pura del screensaver

Función	Entradas	Salidas	Descripción
hsvToRgb	hueDegrees, saturation, value (float)	SDL_Color	Convierte un color HSV a RGB de 8 bits; base de la paleta neón.
pickTronHue	rng (std::mt19937&)	float (grados)	Elige un tono siguiendo la paleta Tron (55% cian, 25% naranja, 13% azul, 7% ámbar) con el generador sembrado.
initBeams	n, width, height (int); length, speedMin, speedMax (float); rng	std::vector<LightBeam>	Crea N líneas con posición, ángulo, velocidad y color pseudoaleatorios.
updateBeam	beam (LightBeam&), dt (float), width, height (int)	(modifica beam)	Fase 1, integra posición y llama a keepInsideBounds; independiente por línea.
keepInsideBounds	beam (LightBeam&), width, height (int)	(modifica beam)	Reacomoda una línea dentro de la ventana, invirtiendo la velocidad solo si aún va hacia la pared.
segmentsIntersect	8 floats, extremos de dos segmentos	bool	Test de intersección de segmentos por orientación (producto cruz).
beamsCollide	a, b (const LightBeam&)	bool	Fase 2, solo lectura; filtro AABB más test de intersección real.
resolveCollision	beamA, beamB (LightBeam&); width, height; speedMin, speedMax, cooldownSeconds	CollisionOutcome	Choque elástico, intercambio de color, separación y reacomodo a bordes.
computeChecksum	beams (const std::vector<LightBeam>&)	uint64_t	Hash FNV-1a de posiciones y colores, para comparar corrección entre versiones.

Tabla A2.1. Catálogo de funciones, neon_lines.

config.h y config.cpp, parámetros y validación

Función	Entradas	Salidas	Descripción
parseArgs	argc, argv; config (Config&); isParallelBinary (bool)	ParseResult	Lee y valida los parámetros de línea de comandos; solicita N por consola si falta.
printUsage	programName; isParallelBinary (bool)	(stdout)	Imprime el mensaje de ayuda y uso.
requestNFromConsole	n (int&)	bool	Pide N por consola, reintentada ante entradas inválidas, sale limpio ante EOF.

Tabla A2.2. Catálogo de funciones, config.

paralelo.cpp, versiones con OpenMP

Función	Entradas	Salidas	Descripción
resolvePairs	beams (vector&); pairs (vector<pair<int,int>>&); config; sparks (puede ser nullptr); collisionCount	(modifica beams)	Ordena los pares por (i,j) y resuelve cada choque; única función que aplica los choques, llamada desde un solo hilo.
buildSpatialGrid	beams; width, height (int); cellSize (float); grid (SpatialGrid&)	(llena grid)	Construye la grilla espacial, formato CSR, usada por v4 para acotar los pares a probar.
applyScheduleSetting	schedule (Schedule)	(omp_set_schedule)	Traduce --schedule a la política de OpenMP usada por los 'omp for' del archivo.

Tabla A2.3. Catálogo de funciones, paralelo.

render.h y render.cpp, dibujo y HUD

Función	Entradas	Salidas	Descripción
BeamGeometry::resize	n (size_t)	(reserva buffers)	Reserva los buffers de vértices e índices para n líneas y precalcula los índices de cada quad, una sola vez.
buildBeamQuad	geometry (BeamGeometry&); index (size_t); beam (const LightBeam&)	(escribe en geometry)	Escribe los 4 vértices (glow y núcleo) de una línea en su posición fija; segura para omp parallel for.
submitBeamGeometry	renderer (SDL_Renderer*); geometry (const BeamGeometry&)	(2 draw calls)	Emite las 2 llamadas a SDL_RenderGeometry (glow y núcleo) para todas las líneas de una vez. Solo hilo principal.
drawGrid	renderer; width, height (int); globalTime (float)	(dibuja)	Rejilla de fondo con un pulso sutil en el tiempo.
updateSparks / drawSparks	sparks (vector<Spark>&); dt (float) / renderer	(actualiza, dibuja)	Vida y dibujo de las chispas que aparecen en cada choque.

Hud::Hud (constructor)	renderer; fontPath (string); windowHeight (int)	(ninguna)	Inicializa SDL2_ttf y abre la fuente; si falla, deja isEnabled()=false y avisa por consola sin detener el programa.
Hud::setText	text (const string&)	(actualiza textura)	Regenera la textura del HUD solo si el texto cambi6 desde la 6ltima llamada, no en cada frame.

Tabla A2.4. Cat6logo de funciones, render.

bench_io.h, bench_io.cpp y main(), utilidades compartidas y punto de entrada

Funci6n	Entradas	Salidas	Descripci6n
createWindowAndRenderer	config (const Config&); windowTitle (const char*); window/renderer (referencias)	bool (6xito)	Crea ventana y renderer con fallback a software; limpia lo creado si falla. Compartida por ambos ejecutables.
appendCsvRow	path; versionLabel; scheduleLabel; n, threads, frames (int); seed; tUpdateMs, tCollisionMs, tRenderMs, tTotalMs, fpsAvg (double); checksum (uint64_t)	(escribe fila)	Agrega una fila de resultados al CSV de la bit6cora, con encabezado si el archivo es nuevo.
main (secuencial.cpp / paralelo.cpp)	argc (int); argv (char*[])	int (c6digo de salida)	Punto de entrada; parsea argumentos, inicializa las l6neas y SDL, corre el loop de frames (Diagrama 2), imprime resultados de benchmark y libera todos los recursos.

Tabla A2.5. Cat6logo de funciones, bench_io y main().

Anexo 3, Bit6cora de pruebas

Fuente de los datos: resultados/bitacora.csv (276 filas, corridas reales en esta maquina), resumido con scripts/analizar.py. La configuracion de la maquina esta en la Tabla 4 de la seccion de Resultados. Casi todas las corridas usaron --seed 42 y --frames 600, con un minimo de 10 repeticiones por configuracion (varias para N=2000 tienen entre 11 y 32, por las corridas extra de schedule y checksum).

El resumen completo esta en resultados/resumen.csv, adjunto junto con el codigo; las tablas 5, 6 y 7 del cuerpo del informe ya extraen de ahi la muestra representativa que sustenta el analisis. La Captura A3.1, mas abajo, es la salida real de scripts/analizar.py sobre nuestra bitacora.

Verificación de corrección, checksum entre versiones (N=2000)

Con la misma --seed 42 y --frames 600, probamos 25 combinaciones de version, hilos y schedule para N=2000; la tabla de abajo muestra una muestra de 10, todas con el mismo checksum. La misma comprobacion se repitio, con resultado identico, para N=100, 500, 1000 y 4000 (ver resultados/bitacora.csv).

Configuración	Checksum
v1, 1 hilo	5186230241032989736
v2, 4 hilos, dynamic	5186230241032989736
v2, 8 hilos, static	5186230241032989736
v3, 4 hilos, dynamic	5186230241032989736
v3, 8 hilos, dynamic	5186230241032989736
v3, 8 hilos, guided	5186230241032989736
v3, 8 hilos, static	5186230241032989736
v3-naive, 8 hilos, dynamic	5186230241032989736
v4, 4 hilos, dynamic	5186230241032989736
v4, 8 hilos, guided	5186230241032989736

Tabla A3.2. Checksum final por configuración, N=2000; ninguna versión paralela introduce condiciones de carrera.

Verificación del rebote en esquinas

Escribimos un programa de prueba independiente que llama a updateBeam en un ciclo y registra vx/vy frame a frame. Un rebote en una sola pared cambia un eje una sola vez, sin repetirse en el siguiente frame; un rebote simultaneo en ambas paredes (esquina real) cambia los dos ejes en un solo frame y la linea sigue su trayectoria limpia, sin rebotes repetidos. Con esto confirmamos que el bug que temblaba/atrapaba una linea en una esquina quedo corregido.

N máximo que sostiene 60 FPS o más (modo ventana, con render real)

Con ventana real (sin --headless) el FPS ya refleja computo y render juntos. Hicimos busqueda manual para v1, v3 y v4 (8 hilos): las tres sostienen 60 FPS o mas hasta un N muy parecido (v1 hasta ~1200, v3 hasta ~1280, v4 hasta ~1310). Que las tres crucen los 60 fps en un rango tan cercano, aun cuando v4 es muchisimo mas barata en computo puro (seccion 5.1), es la Ley de Amdahl en accion: en este rango de N el render (14 a 19 ms por frame) domina sobre cualquier fase paralelizable.

```
./paralelo.exe 1250 --mode v3 --threads 8 --frames 180 --seed 42
```

```
./paralelo.exe 1300 --mode v4 --threads 8 --frames 180 --seed 42
```

Capturas de las mediciones

Captura real de la salida de scripts/analizar.py sobre nuestra bitacora completa (276 filas).

```

C:\WINDOWS\system32\conhost.exe
classic-console-test
PS C:\github\Tron-screensaver> python scripts\analizar.py resultados\bitacora.csv

```

N	version	schedule	hilos	t_total(ms)	fps	speedup	eficiencia	checksum_ok
100	v1-secuencial	n/a	1	0.088	12018.3	1.00	1.000	True
100	v3	dynamic	8	0.764	1343.8	0.11	0.014	True
100	v4	dynamic	8	0.722	1410.8	0.12	0.015	True
500	v1-secuencial	n/a	1	1.018	987.3	1.00	1.000	True
500	v3	dynamic	8	0.868	1174.2	1.17	0.147	True
500	v4	dynamic	8	0.806	1276.5	1.26	0.158	True
1000	v1-secuencial	n/a	1	3.260	306.9	1.00	1.000	True
1000	v3	dynamic	8	1.443	718.5	2.26	0.282	True
1000	v4	dynamic	8	0.808	1255.0	4.03	0.504	True
2000	v1-secuencial	n/a	1	7.724	135.2	1.00	1.000	True
2000	v2	dynamic	4	7.403	135.1	1.04	0.261	True
2000	v2	dynamic	8	6.650	150.4	1.16	0.145	True
2000	v2	guided	4	6.608	151.3	1.17	0.292	True
2000	v2	guided	8	6.653	150.3	1.16	0.145	True
2000	v2	static	4	7.198	138.9	1.07	0.268	True
2000	v2	static	8	6.578	152.0	1.17	0.147	True
2000	v3	dynamic	4	2.633	379.7	2.93	0.733	True
2000	v3	dynamic	8	2.271	447.9	3.40	0.425	True
2000	v3	guided	4	3.952	253.1	1.95	0.489	True
2000	v3	guided	8	2.709	370.1	2.85	0.356	True
2000	v3	static	4	3.638	274.9	2.12	0.531	True
2000	v3	static	8	2.863	351.1	2.70	0.337	True
2000	v3-naive	dynamic	4	3.163	316.1	2.44	0.610	True
2000	v3-naive	dynamic	8	2.411	416.1	3.20	0.401	True
2000	v3-naive	guided	4	4.105	243.6	1.88	0.470	True
2000	v3-naive	guided	8	2.992	334.3	2.58	0.323	True
2000	v3-naive	static	4	4.375	228.6	1.77	0.441	True
2000	v3-naive	static	8	3.310	302.1	2.33	0.292	True
2000	v4	dynamic	4	0.607	1648.3	12.73	3.183	True
2000	v4	dynamic	8	1.051	962.3	7.35	0.918	True
2000	v4	guided	4	0.588	1699.7	13.13	3.282	True
2000	v4	guided	8	0.822	1217.0	9.40	1.175	True
2000	v4	static	4	0.628	1591.5	12.29	3.073	True
2000	v4	static	8	0.682	1467.0	11.33	1.416	True
4000	v1-secuencial	n/a	1	20.497	49.6	1.00	1.000	True
4000	v3	dynamic	8	6.394	157.6	3.21	0.401	True
4000	v3-naive	dynamic	8	7.623	131.2	2.69	0.336	True
4000	v4	dynamic	8	1.559	649.0	13.15	1.643	True

```

Nota: 'checksum_ok' compara el checksum entre las repeticiones de UNA MISMA config;
para validar que la paralelizacion es correcta, compara ademas el checksum de cada
version contra v1 con el mismo --seed y --frames (deben coincidir exactamente).
PS C:\github\Tron-screensaver>

```

Captura A3.1. Salida real de python scripts/analizar.py resultados/bitacora.csv.

Captura de las 10 repeticiones crudas de una misma configuracion (N=2000, v3, 8 hilos, dynamic), mostrando el mismo checksum en cada una.

```

PS C:\github\Tron-screensaver> Get-Content resultados\bitacora.csv | Select-String "^v3,dynamic,2000,8," | Select-Object -First 10

```

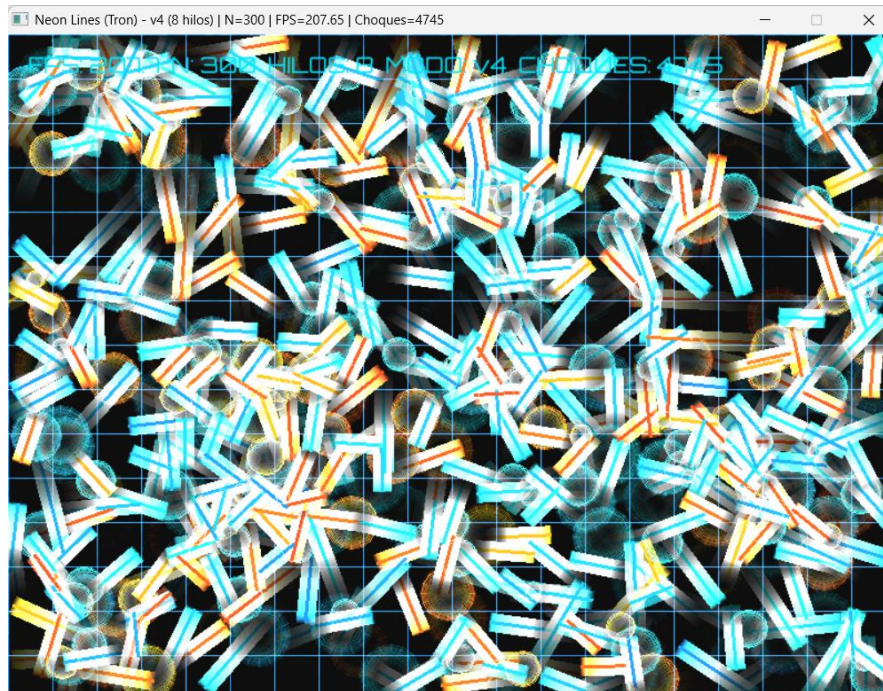
```

v3,dynamic,2000,8,600,42,0.236668,2.45667,0.00333309,2.69667,370.828,5186230241032989736
v3,dynamic,2000,8,600,42,0.206668,2.345,0.2.55167,391.901,5186230241032989736
v3,dynamic,2000,8,600,42,0.23833,2.52667,0.2.765,361.664,5186230241032989736
v3,dynamic,2000,8,600,42,0.446667,2.64,0.3.08667,323.974,5186230241032989736
v3,dynamic,2000,8,600,42,0.236667,2.51333,0.2.75,363.636,5186230241032989736
v3,dynamic,2000,8,600,42,0.214999,2.36,0.2.575,388.349,5186230241032989736
v3,dynamic,2000,8,600,42,0.146664,2.29667,0.00166655,2.445,408.998,5186230241032989736
v3,dynamic,2000,8,600,42,0.250001,2.64333,0.2.89333,345.622,5186230241032989736
v3,dynamic,2000,8,600,42,0.203334,2.25,0.2.45333,407.609,5186230241032989736
v3,dynamic,2000,8,600,42,0.183334,2.37667,0.2.56,390.625,5186230241032989736

```

Captura A3.2. Diez mediciones crudas de resultados/bitacora.csv para una misma configuracion.

Captura real del screensaver corriendo con ventana (no es un mockup): v4, 8 hilos, N=300, con el HUD mostrando FPS, hilos, modo y choques en tiempo real.



Captura A3.3. Neon Lines corriendo en ventana real (v4, 8 hilos, N=300).

Repositorio

El proyecto se encuentra implementado en el siguiente [repositorio](#) (clic en la imagen):

